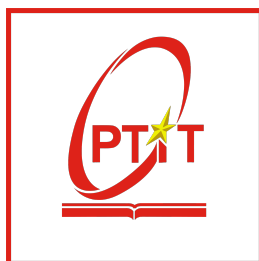


HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA AN TOÀN THÔNG TIN



BÁO CÁO BÀI THỰC HÀNH 1 HỌC PHẦN LẬP TRÌNH WEB ĐỀ TÀI: INFOSOUND

Giao diện nền tảng chia sẻ và nghe nhạc cộng đồng bằng ReactJS

Sinh viên thực hiện: **Chử Đức Anh**

Mã sinh viên: **B23DCAT004**

Học phần: **Lập trình Web (INT1434)**

Hà Nội, ngày 23 tháng 9 năm 2026

Mục lục

1	Tổng quan	2
2	Công nghệ và kiến trúc	2
2.1	Công nghệ sử dụng	2
2.2	Cấu trúc thư mục	3
2.3	Định tuyến và bố cục chung	3
3	Các chức năng đã xây dựng	4
3.1	Trang chủ	4
3.2	Bảng tin	5
3.3	Tìm kiếm	6
3.4	Chi tiết bài hát	7
3.5	Hồ sơ nghệ sĩ	8
3.6	Playlist	9
3.7	Thư viện	10
3.8	Tải lên	11
3.9	Đăng nhập / Đăng ký	12
3.10	Cài đặt	13
4	Một số điểm kỹ thuật	13
4.1	Lỗi đã gặp và cách khắc phục	14
5	Thiết kế giao diện	14
6	Hướng dẫn chạy	14

1 Tổng quan

InfoSound là giao diện frontend cho một nền tảng chia sẻ và nghe nhạc cộng đồng, lấy cảm hứng từ các dịch vụ như SoundCloud. Người dùng có thể nghe nhạc, theo dõi nghệ sĩ, tạo và lưu playlist, bình luận theo từng giây của bài hát và tải lên bản thu của mình. Ứng dụng gồm **10 trang** và một trang 404, được xây dựng hoàn toàn bằng **ReactJS**.

Điểm nhấn của giao diện là **dạng sóng (waveform) tương tác**: bấm vào dạng sóng để tua, di chuột để xem trước thời điểm, và bình luận của người nghe được gắn theo từng giây, hiện thành các chấm màu vàng ngay dưới dạng sóng.

Dữ liệu minh hoạ sử dụng các nghệ sĩ và bài hát EDM có thật (Martin Garrix, Kygo, Gryffin, DEAMN, Alan Walker, Avicii, ILLENIUM, Zedd); số lượt nghe, lượt thích, lượt đăng lại là số giả lập. Người dùng đang đăng nhập trong bản demo là **Chủ Đức Anh** (@chuducanh).

Mục tiêu của bài thực hành:

- Xây dựng giao diện hoàn chỉnh gồm 10 trang bằng ReactJS, có điều hướng giữa các trang.
- Tổ chức mã nguồn theo thành phần (component) dùng lại được, quản lý trạng thái dùng chung bằng Context API.
- Giao diện responsive từ điện thoại đến desktop, hỗ trợ chế độ sáng/tối và điều hướng bằng bàn phím.

2 Công nghệ và kiến trúc

2.1 Công nghệ sử dụng

Thành phần	Phiên bản	Vai trò
React / React DOM	19.3	Thư viện xây dựng giao diện theo component, hooks (<code>useState</code> , <code>useEffect</code> , <code>useMemo</code> , <code>useRef</code>), Context API
React Router DOM	7.18	Định tuyến phía client bằng <code>HashRouter</code> , đọc tham số qua <code>useParams</code> , <code>useSearchParams</code>
Vite	8.3	Máy chủ phát triển có hot reload và công cụ build production
lucide-react	1.47	Bộ biểu tượng dạng SVG (play, heart, repost, ...)
CSS thuần	–	Design token bằng biến CSS, Flexbox/Grid, media query; không dùng framework CSS
oxlint	1.81	Kiểm tra lỗi và quy ước mã nguồn

Bảng 1: Công nghệ và thư viện sử dụng

2.2 Cấu trúc thư mục

Thư mục	Nội dung
components/	Thành phần dùng lại: Header, PlayerBar, Waveform, TrackRow, TrackTile, PlaylistTile, ArtistCard, Tabs, Cover, ...
context/	PlayerContext (trình phát giả lập), LibraryContext (bài đã thích, đăng lại, theo dõi, playlist đã lưu, giao diện sáng/tối)
data/	mock.js: dữ liệu nghệ sĩ, bài hát, playlist, bảng tin, bình luận và các hàm truy vấn
pages/	10 trang chính và trang 404
styles/	tokens.css (màu, font, bo góc), base.css, components.css, pages.css
utils/	format.js: định dạng thời gian, số lượt nghe, sinh dạng sóng theo seed

Bảng 2: Cấu trúc thư mục src/

2.3 Định tuyến và bố cục chung

Ứng dụng dùng HashRouter để khi deploy lên máy chủ tĩnh (ví dụ GitHub Pages) không bị lỗi 404 khi tải lại trang. Trang đăng nhập nằm ngoài bố cục chung; các trang còn lại được lồng trong component Layout thông qua <Outlet />, gồm thanh điều hướng phía trên (logo, menu, ô tìm kiếm, thông báo, menu tài khoản) và thanh phát nhạc cố định phía dưới (phát/tạm dừng, chuyển bài, trộn, lặp, âm lượng).

STT	Trang	Đường dẫn	Component
1	Trang chủ	/	Home.jsx
2	Bảng tin	/feed	Feed.jsx
3	Tìm kiếm	/search?q=	Search.jsx
4	Chi tiết bài hát	/track/:id	Track.jsx
5	Hồ sơ nghệ sĩ	/artist/:id	Artist.jsx
6	Playlist	/playlist/:id	Playlist.jsx
7	Thư viện	/library	Library.jsx
8	Tải lên	/upload	Upload.jsx
9	Đăng nhập / Đăng ký	/auth	Auth.jsx
10	Cài đặt	/settings	Settings.jsx

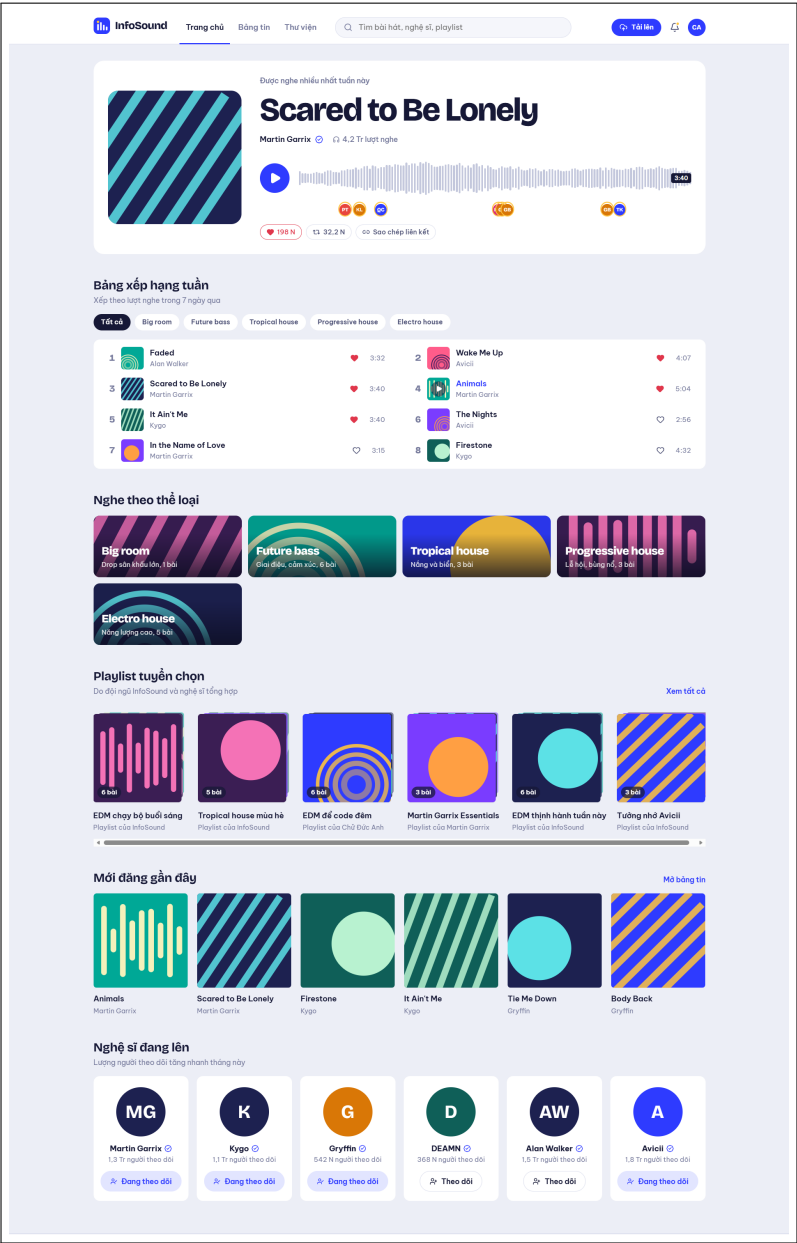
Bảng 3: Danh sách trang và đường dẫn

3 Các chức năng đã xây dựng

3.1 Trang chủ

Hiển thị. Bài hát được nghe nhiều nhất tuần (*Scared to Be Lonely* – Martin Garrix) với ảnh bìa, số lượt nghe và dạng sóng lớn có các chấm bình luận; bảng xếp hạng tuần 8 bài lọc được theo thể loại (Big room, Future bass, Tropical house, Progressive house, Electro house); lưới thể loại, playlist tuyển chọn, bài mới đăng và danh sách nghệ sĩ kèm nút theo dõi.

Công nghệ. Component `Home` dùng `useState` lưu thể loại đang chọn và lọc mảng `chart` ngay khi render. Dạng sóng là component `Waveform` vẽ bằng SVG, số cột tự điều chỉnh theo chiều rộng nhờ `ResizeObserver`. Bố cục bảng xếp hạng và các lưới dùng CSS Grid; liên kết giữa các trang dùng `<Link>` của React Router.

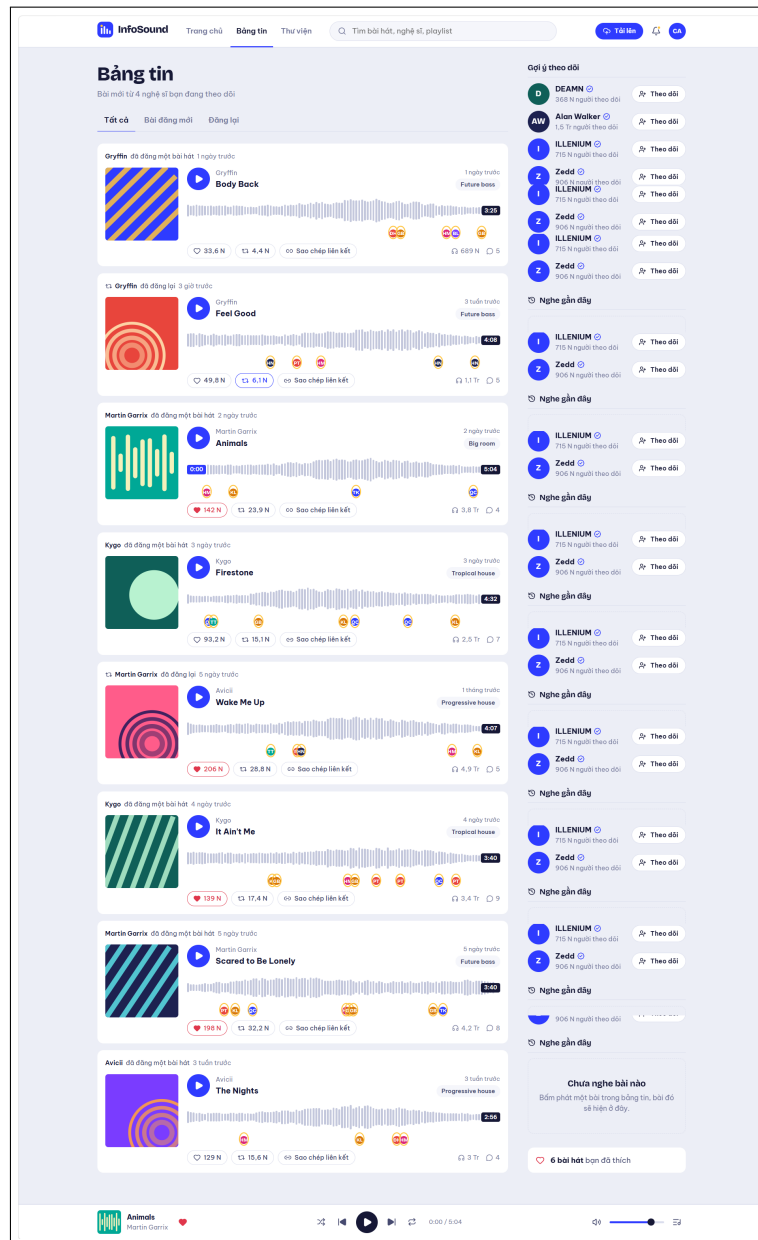


Hình 1: Trang chủ

3.2 Bảng tin

Hiển thị. Đồng thời gian các bài đăng mới và bài đăng lại từ những nghệ sĩ người dùng đang theo dõi (Martin Garrix, Kygo, Gryffin, Avicii), mỗi mục có dạng sóng, nút thích, đăng lại, sao chép liên kết. Tab lọc *Tất cả / Bài đăng mới / Đăng lại*; cột bên phải gợi ý nghệ sĩ nên theo dõi và danh sách nghe gần đây.

Công nghệ. Dữ liệu *feed* được lọc theo tab bằng `useState`; danh sách đang theo dõi lấy từ `LibraryContext`, lịch sử nghe lấy từ `PlayerContext` qua các hook `useLibrary()` và `usePlayer()`. Component `Tabs` và `TrackRow` được dùng lại ở nhiều trang. Bố cục hai cột dùng CSS Grid, tự chuyển thành một cột trên màn hình nhỏ.



Hình 2: Bảng tin

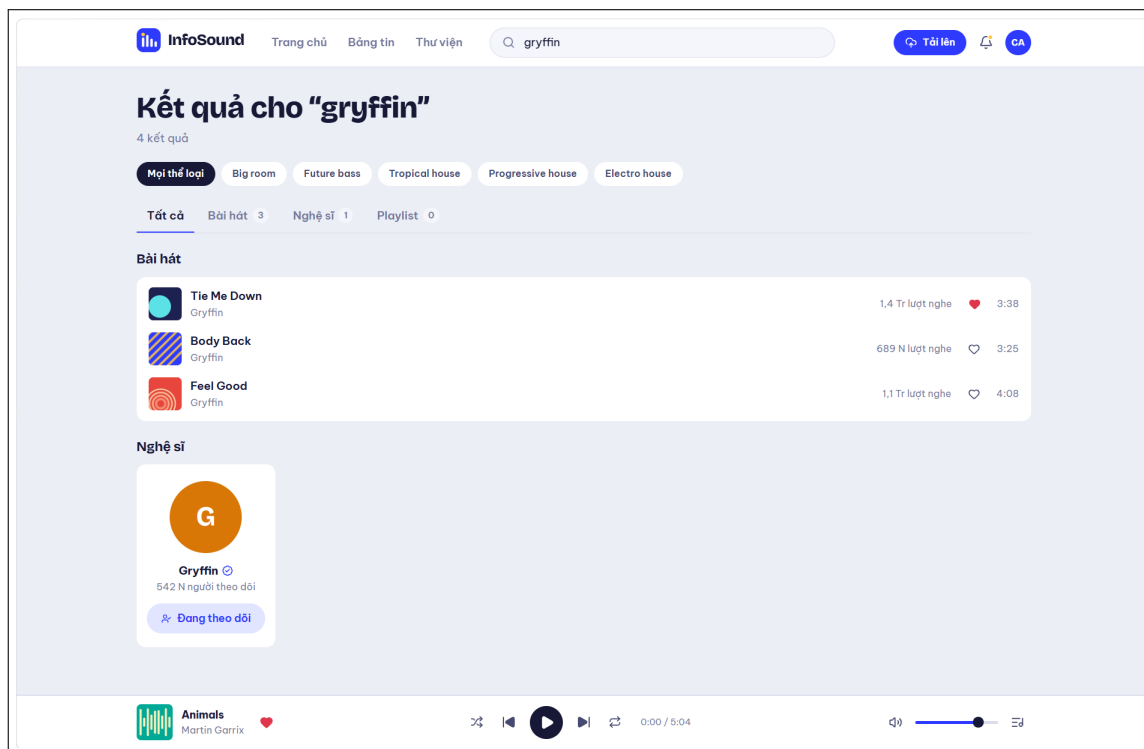
Ghi chú: do trang dài theo chiều dọc nên ảnh chụp toàn trang bằng ứng dụng bị lỗi hiển thị một chút, mong thầy thông cảm :D

3.3 Tìm kiếm

Hiển thị. Kết quả tìm kiếm theo tên bài, nghệ sĩ, thể loại và thể, chia thành các nhóm Bài hát / Nghệ sĩ / Playlist kèm tổng số kết quả; hàng chip lọc theo thể loại. Khi chưa nhập từ khoá, trang hiển thị gợi ý tìm kiếm và lưới thể loại; khi không có kết quả, hiển thị trạng thái trống.

Công nghệ. Từ khoá và thể loại được lưu trên URL bằng `useSearchParams` nên có thể chia sẻ đường dẫn kết quả. Chuỗi được chuẩn hoá bằng `String.normalize('NFD')`

và bỏ dấu, nhờ đó gõ “*không dau*” vẫn tìm được nội dung có dấu. Kết quả được tính trong useMemo để chỉ tính lại khi từ khoá hoặc thể loại thay đổi.

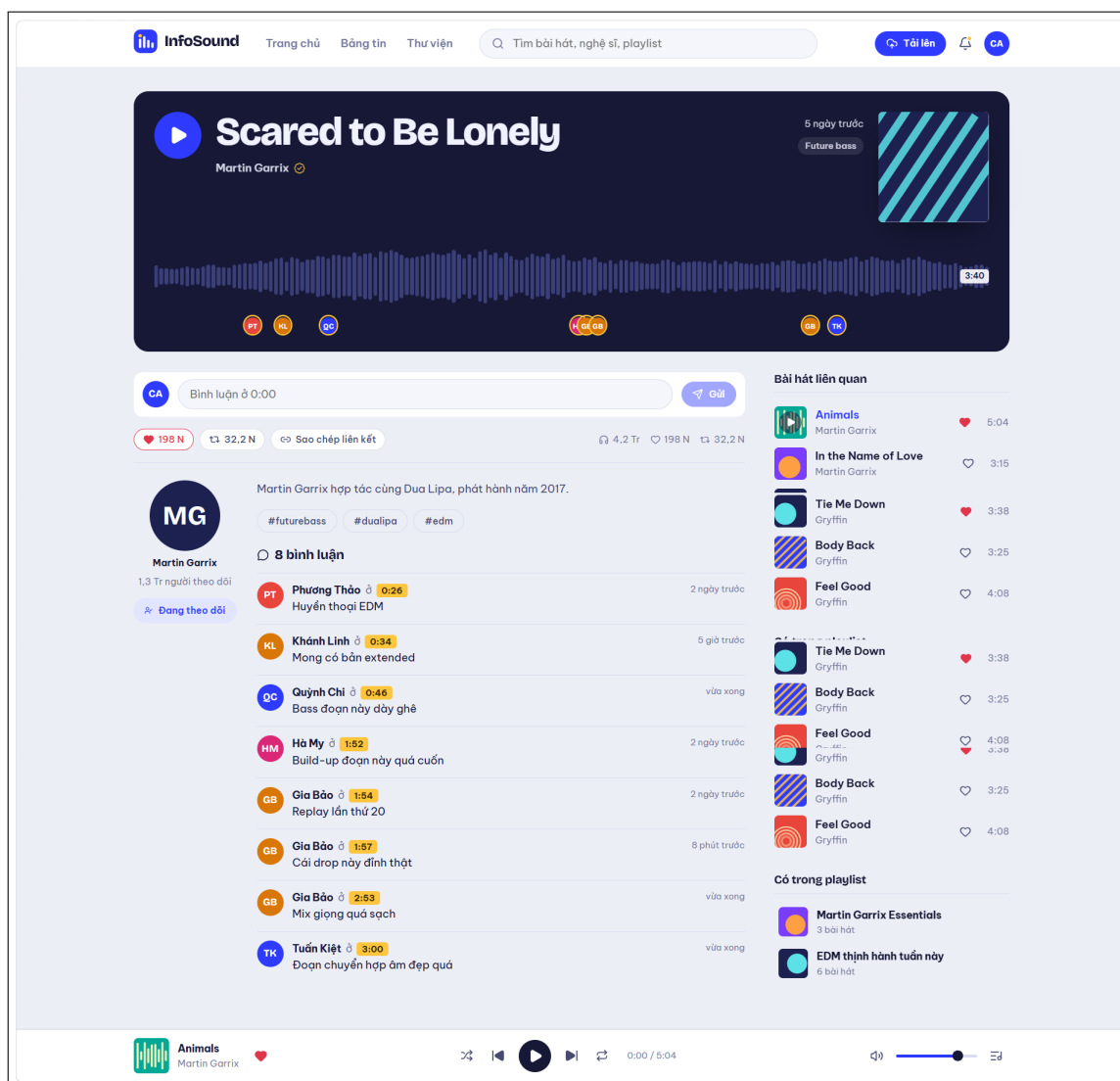


Hình 3: Trang tìm kiếm

3.4 Chi tiết bài hát

Hiển thị. Ảnh bìa, tên bài, nghệ sĩ, thể loại và dạng sóng lớn có các bình luận gắn theo từng giây; ô viết bình luận tại đúng thời điểm đang nghe; thống kê lượt nghe, thích, đăng lại; mô tả, thể, các playlist chứa bài, danh sách bình luận và bài hát liên quan (cùng nghệ sĩ, cùng thể loại).

Công nghệ. Id bài hát đọc từ URL bằng useParams. Bấm vào dạng sóng gọi hàm seek() của PlayerContext để tua; vị trí bình luận được tính theo tỉ lệ giây / thời lượng. Bình luận mới được thêm vào state cục bộ bằng useState; useRef và scrollToView dùng để cuộn đến khung bình luận. Nút sao chép liên kết dùng navigator.clipboard.



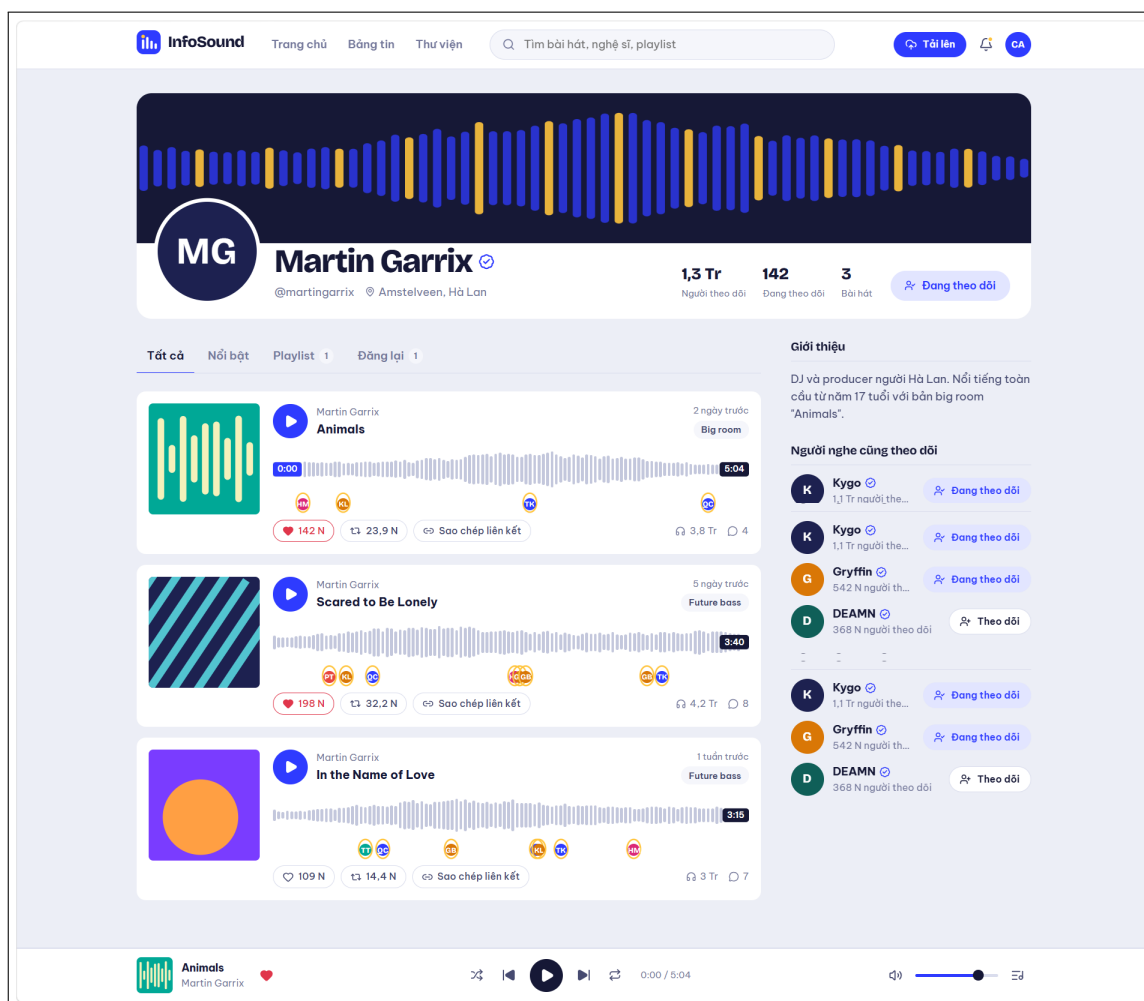
Hình 4: Chi tiết bài hát

Ghi chú: do trang dài theo chiều dọc nên ảnh chụp toàn trang bằng ứng dụng bị lỗi hiển thị một chút, mong thầy thông cảm :D

3.5 Hồ sơ nghệ sĩ

Hiển thị. Ảnh bìa họa tiết dạng sóng riêng của từng nghệ sĩ, ảnh đại diện, huy hiệu xác minh, nơi sống, số người theo dõi, đang theo dõi và số bài hát; nút theo dõi; các tab *Tất cả* / *Nổi bật* / *Playlist* / *Đăng lại*; cột phải là phần giới thiệu và nghệ sĩ tương tự.

Công nghệ. Id nghệ sĩ lấy từ `useParams`, các hàm truy vấn `tracksByArtist`, `playlistsByArtist` lọc dữ liệu tương ứng. Ảnh bìa là SVG sinh từ `makePeaks()` với seed là id nghệ sĩ nên mỗi hồ sơ có họa tiết riêng và cố định. Trạng thái theo dõi được đồng bộ toàn ứng dụng qua `LibraryContext`.

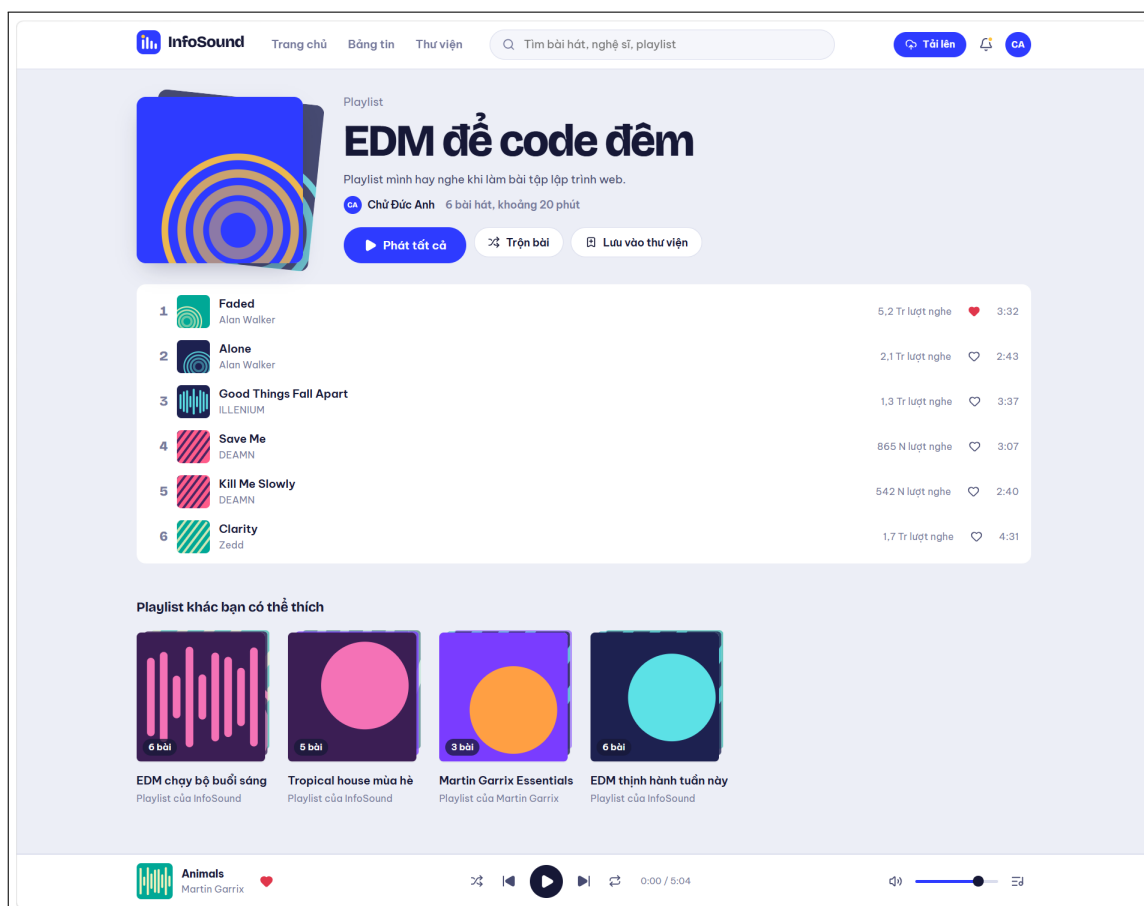


Hình 5: Hồ sơ nghệ sĩ

3.6 Playlist

Hiển thị. Ảnh bìa xếp chồng, tên playlist, mô tả, người tạo, số bài và tổng thời lượng; các nút *Phát tất cả*, *Trộn bài*, *Lưu vào thư viện*; danh sách bài có số thứ tự, lượt nghe, nút thích và thời lượng; phía dưới gợi ý các playlist khác. Hình minh họa là playlist *EDM để code đêm* do người dùng Chử Đức Anh tạo.

Công nghệ. Nút *Phát tất cả* đưa toàn bộ danh sách vào hàng đợi của `PlayerContext`; *Trộn bài* bật chế độ shuffle. Trạng thái đã lưu được bật/tắt qua `toggleSavePlaylist` của `LibraryContext` và lưu vào `localStorage`. Tổng thời lượng được tính bằng `reduce` trên mảng bài hát.

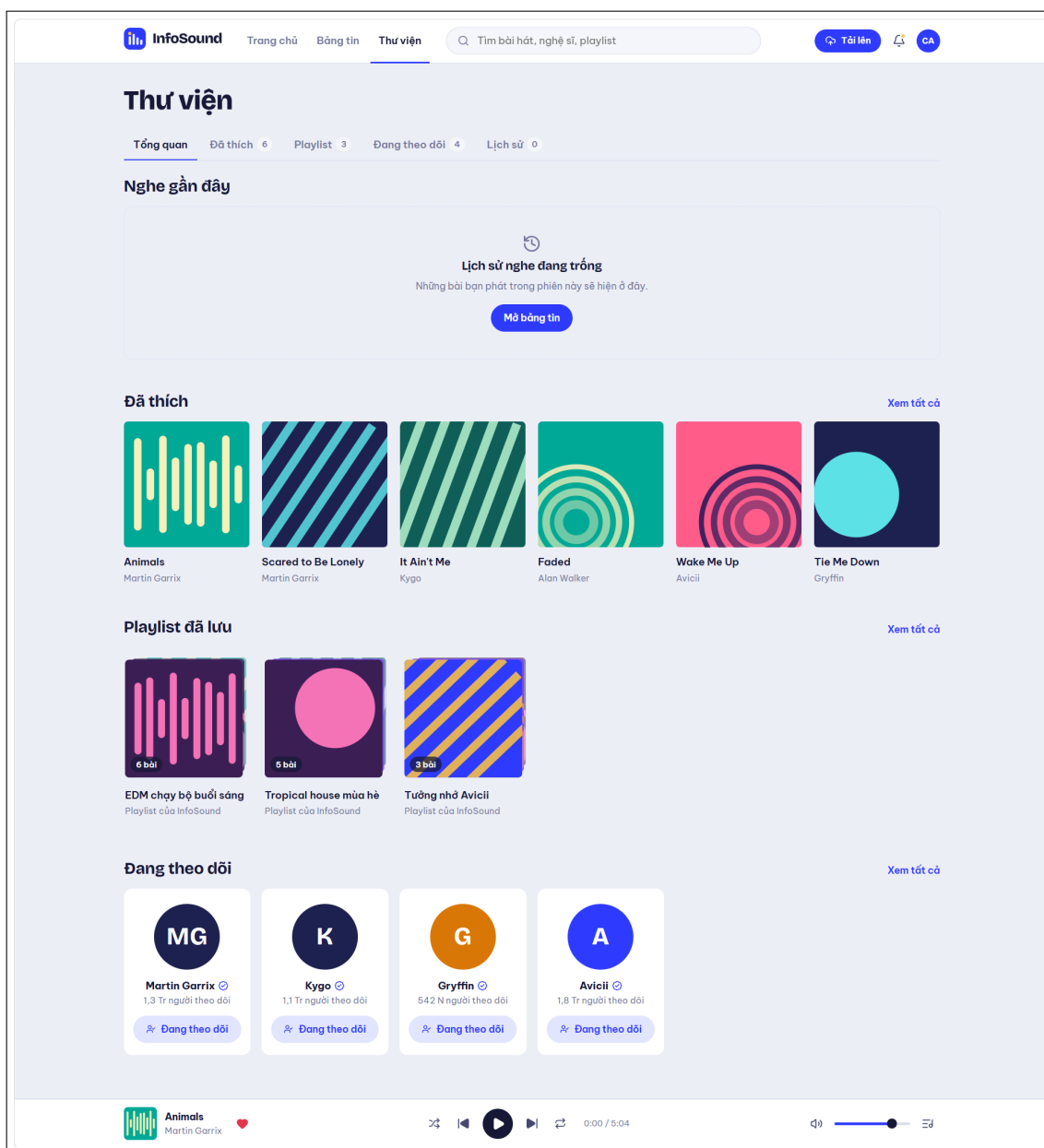


Hình 6: Trang playlist

3.7 Thư viện

Hiển thị. Thư viện cá nhân gồm các tab *Tổng quan* / *Đã thích* / *Playlist* / *Đang theo dõi* / *Lịch sử* kèm số lượng; tab *Tổng quan* hiển thị nhanh bài đã thích, playlist đã lưu, nghệ sĩ đang theo dõi và bài vừa nghe; mỗi tab có trạng thái trống riêng khi chưa có dữ liệu.

Công nghệ. Tab hiện tại lưu trên URL bằng `useSearchParams` (`?tab=likes`), nên có thể mở thẳng một tab. Danh sách đã thích, theo dõi, playlist đã lưu lấy từ `LibraryContext` và được ghi vào `localStorage` trong `useEffect`, nên vẫn giữ nguyên khi tải lại trang; lịch sử nghe lấy từ `PlayerContext`.

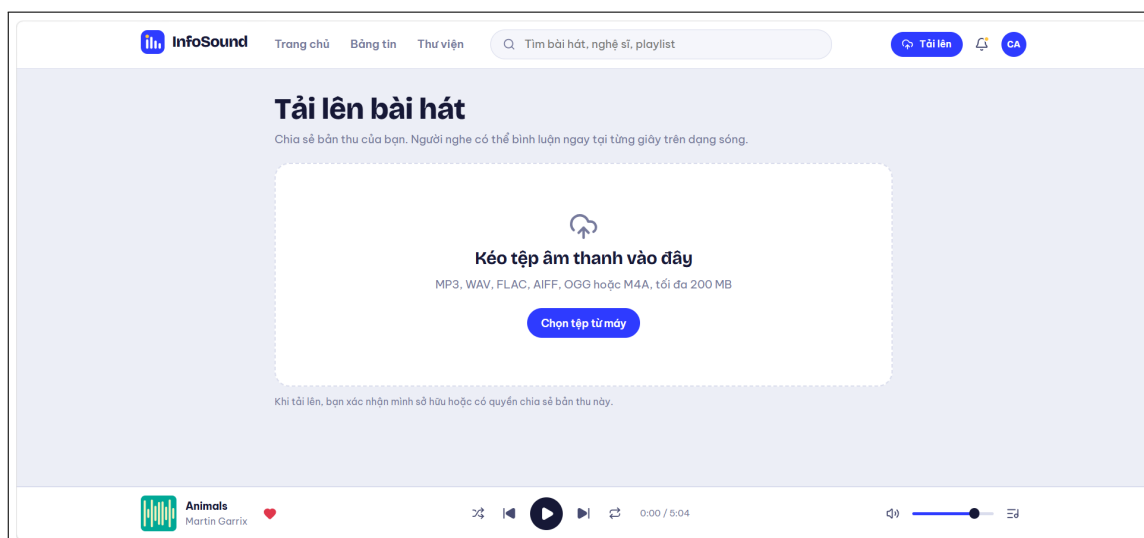


Hình 7: Thư viện

3.8 Tải lên

Hiển thị. Vùng kéo thả tệp âm thanh với hướng dẫn định dạng (MP3, WAV, FLAC, AIFF, OGG, M4A, tối đa 200 MB) và nút chọn tệp từ máy. Sau khi chọn tệp, biểu mẫu nhập tên bài, thể loại, thể (tối đa 5), mô tả và quyền riêng tư (công khai/riêng tư) hiện ra, có thông báo lỗi cho trường bắt buộc và màn hình xác nhận khi tải lên thành công.

Công nghệ. Kéo thả dùng các sự kiện HTML5 `onDragOver`, `onDragLeave`, `onDrop` và `dataTransfer.files`; nút chọn tệp gọi `<input type="file">` ẩn qua `useRef`. Định dạng và dung lượng được kiểm tra phía client trước khi hiện biểu mẫu. Biểu mẫu là controlled component với `useState`, lỗi được thông báo qua `role="alert"`.

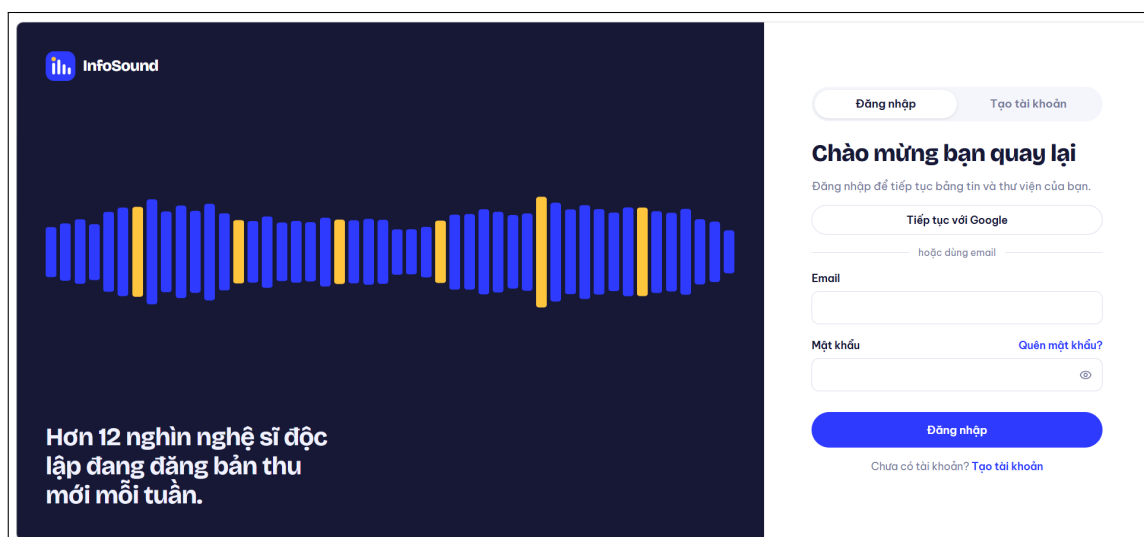


Hình 8: Trang tải lên

3.9 Đăng nhập / Đăng ký

Hiển thị. Màn hình chia đôi: nửa trái là hình minh họa dạng sóng theo nhận diện InfoSound; nửa phải là biểu mẫu với tab chuyển giữa *Đăng nhập* và *Tạo tài khoản*, nút tiếp tục với Google, ô email, mật khẩu có nút ẩn/hiện, liên kết quên mật khẩu và thông báo lỗi dưới từng ô.

Công nghệ. Chế độ và dữ liệu nhập được quản lý bằng `useState`. Email kiểm tra bằng biểu thức chính quy, mật khẩu yêu cầu tối thiểu 8 ký tự; biểu mẫu tắt kiểm tra mặc định của trình duyệt (`noValidate`) để hiển thị lỗi tiếng Việt thống nhất. Đăng nhập hợp lệ chuyển về trang chủ bằng `useNavigate`. Trang này nằm ngoài `Layout` nên không có thanh điều hướng và trình phát.

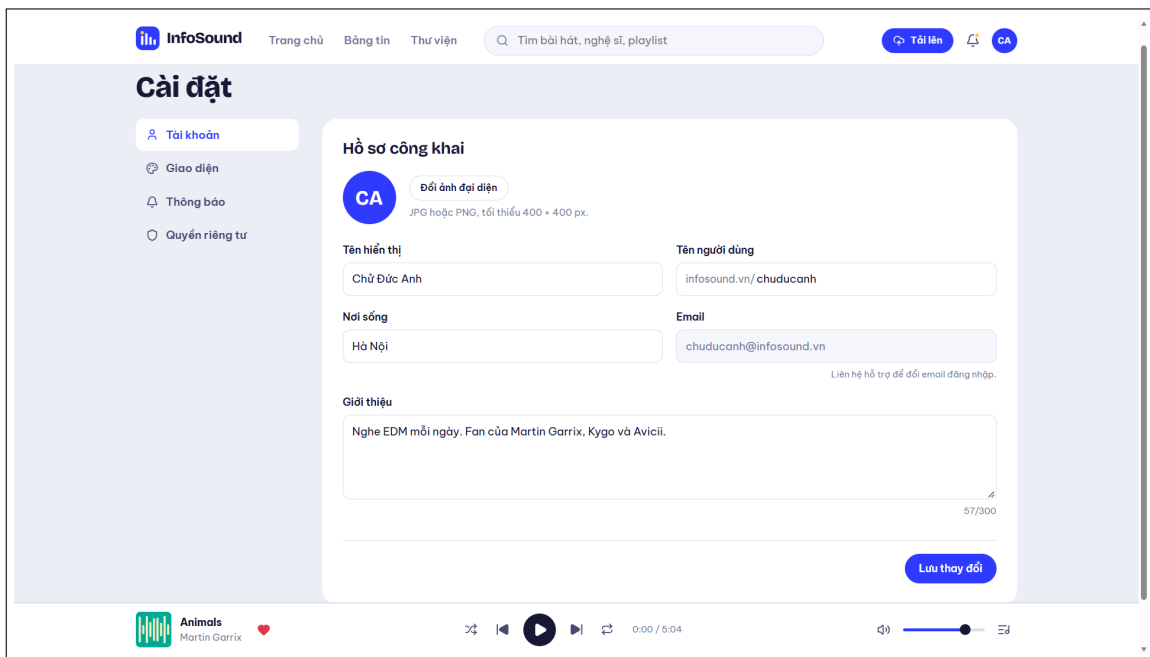


Hình 9: Trang đăng nhập / đăng ký

3.10 Cài đặt

Hiển thị. Menu bên trái gồm *Tài khoản / Giao diện / Thông báo / Quyền riêng tư*. Mục Tài khoản hiển thị hồ sơ công khai của người dùng Chữ Đức Anh: ảnh đại diện, tên hiển thị, tên người dùng, nơi sống, email (khóa, không sửa được) và phần giới thiệu có bộ đếm ký tự. Các mục còn lại là lựa chọn giao diện Sáng/Tối/Theo hệ thống và các công tắc bật tắt thông báo, quyền riêng tư.

Công nghệ. Chế độ giao diện lưu trong `LibraryContext`, áp dụng bằng thuộc tính `data-theme` trên thẻ `<html>`; chế độ theo hệ thống lắng nghe `window.matchMedia('(prefers-color-scheme: dark)')`. Toàn bộ màu sắc là biến CSS nên đổi giao diện không cần tải lại trang. Khi lưu, thông báo *Đã lưu thay đổi* hiện ra và tự ẩn sau 2,2 giây nhờ `setTimeout` trong `useEffect`.



Hình 10: Trang cài đặt

4 Một số điểm kỹ thuật

- **Trình phát giả lập:** không phát âm thanh thật; tiến độ tăng mỗi giây bằng `setInterval` trong `PlayerContext`, nhờ đó dạng sóng, thanh phát và bình luận theo thời gian phản hồi như một trình phát thật. Hỗ trợ hàng đợi, trộn bài, lặp lại, âm lượng và lịch sử nghe.
- **Ảnh bìa và dạng sóng sinh theo seed:** dùng bộ sinh số ngẫu nhiên có seed từ id của bài hát/nghe sĩ, vẽ bằng SVG, nên không cần tải ảnh bên ngoài và kết quả giống nhau ở mọi lần tải.
- **Quản lý trạng thái:** hai Context tách biệt – `PlayerContext` cho việc phát nhạc, `LibraryContext` cho dữ liệu cá nhân – giúp mọi trang dùng chung trạng thái mà không phải truyền props qua nhiều cấp.
- **Responsive và trợ năng:** bố cục cơ giản qua các mốc 1080px, 860px, 600px; có liên kết *Bỏ qua đến nội dung*, thuộc tính `aria-*`, `role` cho tab, slider, switch

và tôn trọng `prefers-reduced-motion`.

4.1 Lỗi đã gặp và cách khắc phục

Khi chạy `npm run dev`, trang hiển thị trắng và console báo lỗi `Uncaught TypeError: destroy is not a function`. Nguyên nhân là effect trong `Layout.jsx` được viết dạng rút gọn:

```
useEffect(() => window.scrollTo(0, 0), [pathname]);
```

Arrow function rút gọn tự trả về kết quả của `window.scrollTo()`; trên các phiên bản Chromium/Edge mới, hàm này trả về một `Promise`. React coi giá trị trả về của effect là hàm dọn dẹp (cleanup), và ở chế độ phát triển `StrictMode` gọi cleanup ngay nên ứng dụng bị lỗi. Cách sửa là đặt thân hàm trong ngoặc nhọn để effect không trả về gì:

```
useEffect(() => {  
  window.scrollTo(0, 0);  
}, [pathname]);
```

5 Thiết kế giao diện

- **Màu sắc:** xanh ultramarine `#2E3BFF` cho tín hiệu (đang phát, nút chính); vàng băng cassette `#FFC53D` cho bình luận và điểm nhấn; nền xám lam `#ECEE66`.
- **Phông chữ:** Bricolage Grotesque cho tiêu đề, Be Vietnam Pro cho nội dung (hỗ trợ đầy đủ tiếng Việt), tải từ Google Fonts.
- **Bố cục:** tham khảo các mẫu thiết kế nền tảng âm nhạc trên Figma Community và các dịch vụ nghe nhạc trực tuyến, sau đó thiết kế lại với nhận diện riêng.

6 Hướng dẫn chạy

Yêu cầu Node.js 20 trở lên.

```
npm install  
npm run dev      # mở http://localhost:5173  
npm run build    # build bản production vào thư mục dist/
```